Agentic AI: hello world to enterprise

Course syllabus · version 2 · approved 2026-10-20

Who it's for: Developers who can write a little Python and call a web API; no machine-learning background assumed. The course uses the Python 3.12 standard library and the official SDKs.

What you'll be able to do: Build an agent that calls tools safely, grounds its answers in documents, remembers the right things, runs as a service and survives failures; know the enterprise practice (evals in CI, tracing, security, durable runs, tenancy, cost, governance) well enough to build it, operate it and pass a developer interview.

38 lessons · about 15 hours

Interview questions & projects: Interview questions (interview/INTERVIEW_QUESTIONS.pdf) · Interview answer key (instructors only) (interview/ANSWER_KEY.pdf) · Basic projects (projects/BASIC_PROJECTS.pdf) · Intermediate projects (projects/INTERMEDIATE_PROJECTS.pdf) · Advanced projects (projects/ADVANCED_PROJECTS.pdf)

Tools you'll use

Tool	What for	Lessons	How you can run it
Python 3.12	the language for every lesson and the app	01-37	local: Python 3.12 (default)
Gemini API (Vertex AI)	default model for the agent in most lessons	02-37	Gemini API key (default) / Vertex AI project and location
Anthropic API (Claude Haiku 5.5)	the second vendor for the wire-format and SDK comparisons	05, 20	recorded replay (default, no key) / Anthropic API key
OpenAI API	the third vendor for the wire-format and SDK comparisons	05, 20	recorded replay (default, no key) / OpenAI API key
llm_replay (vcrpy)	records each model call once and replays it with no key	all model lessons	local: recorded replay (default)
Ollama (local model)	a free small model for the token and sampling lessons	03	local: Ollama (default, free, gemma3:1b under 1 GB download)
pydantic	structured output and the data models of the app	04, 05, 07, 17, 24	local
SQLite (standard library)	memory, the CRM and the audit log	08, 15, 16, 18, 31	local
LangGraph (with SQLite checkpointer)	the agent as a graph with state and interrupts	19, 20, 23	local
MCP Python SDK	exposing the tools as an MCP server and connecting a client	21, 26, 30	local: stdio and Streamable HTTP on localhost
A2A SDK	one agent calling another with an agent card	27	local: two localhost servers
FastAPI + uvicorn	the API of the app and the served agent	34, 37	local
Docker + Compose	the three-replica fleet, Temporal and the capstone	34, 35, 37	local: Docker Compose (needs-docker)
Temporal (temporalio)	durable runs that survive a worker kill	35	local: Temporal dev server in Compose (needs-docker)
OpenTelemetry SDK	traces of model and tool calls	29, 37	local: JSON-lines exporter (default) / local: Phoenix in pip (optional)
Chroma (embedded)	the vector store for RAG	17, 18	local: embedded in-process (default)

Tool	What for	Lessons	How you can run it
numpy	cosine similarity written and checked against the library	17	local
tenacity	retries with backoff and jitter	02, 32	local
pytest	the eval harness and tests	08, 22, 28, 37	local
Authlib (local dev auth server)	OAuth for the MCP walkthrough	26	local: small token issuer written with Authlib

You'll be asked which option you want, and for any keys, just before the lessons that need it.

Before you start

• **OLLAMA_BASE_URL** — Ollama server for lesson 03 (a free local model). Default is the server on this machine; change it only if Ollama runs elsewhere.
• Docker installed (some lessons start services in containers)
Put the values in this course's .env file (template: .env.example).

Contents

Basic — 10 lessons · ~4 h

Chapter 1. Foundations: the first model call and the tool loop

01 Why agentic AI, and what an agent is (and is not) · theory — Tell an agent from a chatbot and from a fixed workflow, and say when not to build an agent.

02 Your first model call: messages, usage, cost and streaming — Make a model call with the official SDK, read the usage and cost, and stream the answer.

03 Tokens, context windows and sampling dials — Count tokens, price a call, and see what the context window and the sampling controls do (and which models lock them).

04 Prompting for agents and structured output with Pydantic — Write a system prompt for an agent and get validated JSON back from the model, with a retry when it is invalid.

05 The tool-call round trip on three vendor wire formats ★ — Declare a tool, let the model ask for it, run it and return the result, in the Claude, OpenAI and Gemini formats.

Chapter 2. Core: an agent you can test and secure

06 Build the agent loop yourself, with step and cost caps — Write the thought-action-observation loop with a step cap, a spend cap and repeated-call detection.

07 Writing good tools: names, schemas and errors the model can act on — Turn Python functions into tools the model uses correctly, and make tool errors something the model can recover from.

08 Your first eval: a golden set and a test that needs no model — Write the first 10 of the 30 golden tasks, with expected end states, and a pytest that scores a scripted fake agent.

09 Safety basics: untrusted tool output, least privilege and approval for writes — Show how a ticket can hijack an agent, then limit what each tool can do and flag every write for approval (the gate itself is lesson 14).

10 Checkpoint: the order-lookup assistant (app v1) ★ — Ship a Basic-level agent with tools, caps, a golden set and safety checks, and run it with one command.

Intermediate — 13 lessons · ~5 h

Chapter 3. Practical: workflows, memory and retrieval

11 Why workflows come before autonomy: chaining, routing and parallel calls — Compare a fixed pipeline with an agent on cost, reliability and debuggability, then build the three basic workflow shapes.

12 Orchestrator-workers and evaluator-optimizer loops — Split one task across worker calls with an orchestrator, and add a judge that sends work back for a second pass.

13 ReAct, plan-and-execute and reflection: which fits and what each costs · walkthrough — Choose a control pattern for a task from its cost, latency and failure modes, and see why reflection is used sparingly.

14 A human approval gate with interrupt and resume — Pause a refund before it is paid, wait for a person, and resume the same run safely.

15 Short-term memory and context engineering: trim, summarise, compact, cache — Keep a long conversation inside the window: trim, summarise and compact, and cache the stable prefix.

16 Long-term memory in SQLite: what to remember, forgetting and privacy — Store customer facts the agent can recall later, with deletion and a rule for what must never be stored.

17 RAG basics: embeddings, chunking and cosine top-k — Embed policy chunks, retrieve the top-k by cosine similarity, and answer with the retrieved text in the prompt.

18 Hybrid search, reranking and agentic RAG with citations ★ — Combine keyword and vector search, let the agent decide when to search again, and cite the section it used.

Chapter 4. Advanced build: LangGraph, MCP and the eval loop

19 Why a graph: the loop as state, nodes, edges and checkpoints — Rebuild the support loop as a LangGraph graph and see what state, checkpoints and interrupts give you that a loop does not.

20 Vendor agent SDKs, LangGraph or a hand loop: how to choose · walkthrough — Compare one vendor SDK with LangGraph and the hand loop on the same task, then pick one with reasons.

21 MCP: build a server and connect a client — Expose the order tools as an MCP server, connect a client to it, and call a tool over the protocol.

22 Eval depth: error analysis, LLM-as-judge and trajectory grading — Find the failing component with error analysis, grade a free-text answer with a judge, and check the judge against human labels.

23 Checkpoint: the support desk with memory, approval and a graph (app v2) ★ — Run the Intermediate support desk: retrieval, memory, a refund approval and a LangGraph graph, scored by the eval.

Advanced — 15 lessons · ~6 h

Chapter 5. Production: multi-agent, protocols, evals in CI and security

24 Why multi-agent, and when a single agent is better — Measure what a second agent costs in tokens and latency, and use one only when isolation or parallel work pays.

25 Multi-agent failure modes: loops, lost context and guards — Reproduce a loop between two agents and add the guards that stop it: budgets, handoff limits and a shared state check.

26 MCP auth and a gateway: what it adds in front of tools · walkthrough — Walk through OAuth for an MCP server, tool poisoning, and what a gateway adds, with a local dev authorization

server.

27 A2A: agent discovery and agent cards — Publish an agent card, let another agent discover it at the well-known URL, and read its skills and auth requirements before sending a task.

27a A2A: the task lifecycle, streaming and delegating to another agent — Send a task to a remote agent, follow its states (submitted, working, input-required, completed, failed, canceled), stream its updates, and handle a failure.

28 Evals in CI: a regression gate that blocks a bad change — Put the golden set in CI, fail the build when success drops, and roll out a model change behind a canary.

29 Observability: traces, what to log, and a dashboard — Trace every model and tool call as spans, log what is safe to log, and build a dashboard for success, cost and latency.

30 Security: an injection test set, sandboxed code and least privilege — Attack the support agent with a test set of injected tickets, then fix it with least privilege and a sandboxed code tool.

31 Guardrails, policy checks, agent identity and an audit log ★ — Add a policy check before each tool call, give each agent its own identity, and write an audit log that answers who did what.

Chapter 6. Enterprise: reliability, cost, serving, durable runs and governance

32 Reliability: retries, timeouts, idempotency and rate limits — Make the refund tool safe to retry: backoff with jitter, timeouts, idempotency keys, and a token-bucket limiter.

33 Cost and latency: caps, caching, routing and batch — Price a ticket, cap its spend, cut cost with caching, routing and batch calls, and check the latency budget.

34 Serving and deployment: FastAPI, Docker, a three-replica fleet and a load test — Serve the agent as an API in a container, run three replicas behind a gateway, load-test it, and roll out a new version safely.

35 Durable execution: checkpoints, Temporal and a worker-kill drill — Kill the worker in the middle of a refund run, restart it, and check that the run finishes once and only once.

36 The enterprise map: tenancy, managed runtimes, governance and HA/DR · walkthrough — Place the agent in an organisation: tenant isolation, managed runtimes, governance, backups and restore objectives.

37 Capstone: the support desk as a production service with an eval gate and a drill ★ — Ship the support desk as a tested, traced, budget-capped service with a CI eval gate, and pass a kill drill on the Compose fleet.

★ = project lesson

Industry reading

How companies that run this at scale do it: one to three posts per chapter.

Ch 1 · Building effective AI agents — Anthropic, 2024-12. Workflows versus agents and the simple augmented-LLM loop that the first chapter builds.
https://www.anthropic.com/engineering/building-effective-agents

Ch 1 · Writing effective tools for agents — with agents — Anthropic, 2025-09. How to design, describe and evaluate the tools your loop calls.
https://www.anthropic.com/engineering/writing-tools-for-agents

Ch 2 · Beyond permission prompts: making Claude Code more secure and autonomous — Anthropic, 2025-10. How sandboxing and permission boundaries make an agent safer to run unattended.
https://www.anthropic.com/engineering/claude-code-sandboxing

Ch 2 · Background Coding Agents: Predictable Results Through Strong Feedback Loops (Honk, Part 3) — Spotify, 2025-12. How verifiers and feedback loops make an agent's output testable and trustworthy.
https://engineering.atspotify.com/2025/12/feedback-loops-background-coding-agents-part-3

Ch 3 · Introducing Contextual Retrieval — Anthropic, 2024-09. A concrete technique that cuts retrieval failures in RAG, with measured results.
https://www.anthropic.com/news/contextual-retrieval

Ch 3 · Effective context engineering for AI agents — Anthropic, 2025-09. What to keep in, fetch just in time or compact out of the window as memory grows.
https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents

Ch 4 · Introducing the Model Context Protocol — Anthropic, 2024-11. The original motivation and design of MCP, the protocol behind chapter 4's tool servers.
https://www.anthropic.com/news/model-context-protocol
Ch 4 · Code execution with MCP: Building more efficient agents — Anthropic, 2025-11. Why many MCP tools bloat context and how code execution reduces tokens.
https://www.anthropic.com/engineering/code-execution-with-mcp
Ch 4 · Code Mode: the better way to use MCP — Cloudflare, 2025-09. An alternative way to call MCP tools through generated code, with the reasoning behind it.
https://blog.cloudflare.com/code-mode/
Ch 5 · How we built our multi-agent research system — Anthropic, 2025-06. Orchestrator-worker design, token cost and lessons from a real multi-agent system.
https://www.anthropic.com/engineering/multi-agent-research-system
Ch 5 · Announcing the Agent2Agent Protocol (A2A) — Google, 2025-04. Why agent-to-agent interoperability needs a protocol and how A2A is shaped.
https://developers.googleblog.com/en/a2a-a-new-era-of-agent-interoperability/
Ch 5 · Demystifying evals for AI agents — Anthropic, 2026-01. Graders, pass@k versus pass^k and how to start an eval suite that gates releases.
https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents
Ch 6 · 1,500+ PRs Later: Spotify's Journey with Our Background Coding Agent (Honk, Part 1) — Spotify, 2025-11. Running a coding agent at company scale: results, failures and operating lessons.
https://engineering.atspotify.com/2025/11/spotifys-background-coding-agent-part-1
Ch 6 · Effective harnesses for long-running agents — Anthropic, 2025-11. How agents keep progress and clean state across many sessions, the basis of durable runs.
https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents
Ch 6 · AI Gateway is generally available: a unified interface for managing and scaling your generative AI workloads — Cloudflare, 2024-05. Caching, rate limiting, retries and analytics in front of model calls for cost and reliability.
https://blog.cloudflare.com/ai-gateway-is-generally-available/

Where next

Forward deployed engineer · role · established

An engineer embedded with customers who scopes, builds and ships LLM/agent solutions on the customer's data and systems. Why next: This course teaches the part an FDE does first: wire a model to a customer's tools with a loop you can explain and test, then prove it with evals and budgets.

- Scoping an agent use case with a customer (is it a workflow or an agent?)
- Evals on customer data and acceptance criteria
- Integrating with customer systems safely (least privilege, approvals)
- Deployment, data-retention and security reviews
- Explaining cost, latency and failure modes to non-engineers

Start it: /tutorial "Forward deployed engineering for AI" basic-v2

Context engineering · discipline · growing

Deciding what goes into the model's window, in what order, and what is fetched just in time, compacted or left out. Why next: Lesson on context windows and the agent loop shows the window filling with tool results; this discipline is how production agents keep quality as the window fills.

- Context rot and attention budget
- Just-in-time retrieval with lightweight identifiers
- Compaction, tool-result clearing and structured note-taking
- Sub-agents with clean contexts
- Prompt caching layout (stable prefix first)

Start it: /tutorial "Context engineering for agents" basic-v2

Agent evaluation and observability platforms · technology · growing

Tools and practice for grading agent runs (outcome and trajectory), tracing every model and tool call, and gating releases on evals. Why next: The foundations course ends with a first golden-set eval and a run log; platforms turn those into dashboards, CI gates and production monitoring (track 6).

- Task, trial, grader, transcript, outcome vocabulary
- pass@k vs pass^k
- LLM-as-judge calibration against human labels
- OpenTelemetry GenAI conventions and tracing back ends
- Evals in CI for every prompt or model change

Start it: /tutorial "Agent evaluation and observability" basic-v2

Harness engineering · discipline · emerging

Designing the scaffolding around a model: system prompt, tools, context delivery, verification loops, memory, sandboxes, so an agent can finish real tasks. Why next: The 150-line loop you write here is a minimal harness; harness engineering is what that loop grows into for long-running work.

- Initializer + incremental coding-agent pattern
- Progress files and clean committed state between sessions
- Verification (tests, browser automation) inside the loop
- Permissions and sandboxing
- Which harness parts become unnecessary as models improve

Start it: /tutorial "Harness engineering" basic-v2

Loop engineering · discipline · emerging

Designing the systems that prompt coding agents repeatedly (goal loops, cron or event triggers, fresh context per item) instead of prompting by hand. Why next: You build the inner loop (model, tools, stop) here; loop engineering is the outer loop around it, with success criteria and budgets.

- Ralph-style one-item-per-iteration loops with success criteria
- Triggers and schedules (CI failures, new tickets, nightly runs)
- Stop conditions and token budgets for unattended runs
- Fresh context per iteration to limit context rot
- Cost control at API prices

Start it: /tutorial "Loop engineering" basic-v2

Agent protocols (MCP) · technology · growing

Standard ways to expose tools and data to any agent (Model Context Protocol) instead of one-off function schemas per app. Why next: You hand-wrote tool schemas and a registry; MCP is the shared version of that contract (track 4 of this roadmap).

- Host, client and server roles
- Tools, resources, prompts
- stdio vs Streamable HTTP
- Authorization and tool-poisoning risks
- Code Mode: models writing code against MCP tools

Start it: /tutorial "Agentic AI protocols MCP A2A" basic-v2

Appendix: details for reviewers

How this syllabus was checked, and other ways to take it. Learners can skip this part.
Running example — Brightside Outfitters support desk: Everyone has chased a late parcel, so the support desk needs no explanation; refunds make approval, idempotency and injection real problems. order lookup agent (chapter 1) -> RAG, memory and a refund approval in a LangGraph graph (chapters 3-4) -> multi-agent with evals in CI, traces and security (chapter 5) -> reliable, cheap, served and durable with tenants (chapter 6)
LLM use: the course is about agents built on models; every model call is recorded once with a key and replayed without one Recorded once and replayed (no key needed by learners); at most 5 model calls per lesson.

Shorter paths through the same course

Path	Lessons	Time
Essentials	34	~14 h
Professional	38	~15 h
Complete	38	~15 h

Coverage report

67 concepts from 24 independent sources. Must-cover: 55; taught: 55; left out on purpose: 0.

Developer coverage (D1–D18)

Item	Lessons
D1 Purpose and architecture	01, 36
D2 Mental model and core concepts	01, 02, 03
D3 Setup and first working app	02, 10
D4 Day-to-day API surface	02, 04, 05
D5 Data, schemas and configuration	04, 16
D6 Integration with the app stack	21, 34
D7 Async, concurrency and performance basics	33, 34
D8 Errors, validation and reliability	04, 32
D9 Testing and evaluation	08, 22, 28
D10 Debugging and observability	06, 29
D11 Security	09, 30, 31
D12 Packaging, deployment and operations	10, 34
D13 Scaling and cost	33, 34
D14 Versions, upgrades and migration	02, 21
D15 Ecosystem and choosing	20
D16 Common pitfalls and anti-patterns	06, 25
D17 Reference architectures and real use cases	36
D18 End-to-end capstone	37

Enterprise depth coverage

Item	Lessons
1 distributed architecture internals	35
2 cluster operations	34
3 failure and recovery	35
4 extreme scale	33
5 deployment options	34
6 performance engineering at scale	34

Item	Lessons
7 high availability and disaster recovery	36
8 zero-downtime change	34
9 multi-tenancy and isolation	36
10 security and compliance at scale	36
11 cost engineering	33
12 platform and organisation patterns	36
13 frontier	36
14 case studies	36

Out of scope

Kubernetes labs — its own course; the fleet runs on Docker Compose (rule: no Kubernetes labs here)
Transformer internals, Mamba and Titans — research level; papers go to Go deeper

Syllabus history

Version	Date	Changes
1	2026-10-20	first approval
2	2026-10-20	stack metadata: google.genai module, llm_replay without vcr module, Ollama module and gemma3:1b size